Working with temporal and statistical averages

Laboratory 4, DEDP

Table of contents

1 Objective **1**

2 Theoretical aspects **1**

2.1 Statistical mean, variance, and standard deviation 1

2.2 Temporal mean, variance, and standard deviation 2

2.3 Computing averages in Matlab 3

2.4 Loading variables from a file 4

2.5 Summary: functions and notation you should know 4

3 Theoretical Exercises **6**

4 Matlab Exercises **6**

5 Final questions **8**

1 Objective

Use `mean()`, `std()`, and `var()` to summarize real measurements. Distinguish averages along time from averages across observations, choose the correct matrix dimension, and explain why an average of ratios differs from a ratio of totals.

2 Theoretical aspects

2.1 Statistical mean, variance, and standard deviation

Statistical averages are defined using the probability distribution, rather than a particular measured sequence.

For a continuous random variable A with probability density $w_A(x)$, its **statistical mean** is

$$\mu_A = \mathrm{E}\{A\} = \overline{A} = \int_{-\infty}^{\infty} x\, w_A(x)\, dx.$$

The **average squared value** (second moment) is the mean of the squared variable:

$$\mathrm{E}\{A^2\} = \overline{A^2} = \int_{-\infty}^{\infty} x^2\, w_A(x)\, dx.$$

Its **statistical variance** is:

$$\sigma_A^2 = \mathrm{E}\{(A-\mu_A)^2\} = \overline{(A-\mu_A)^2} = \int_{-\infty}^{\infty} (x-\mu_A)^2 w_A(x)\, dx.$$

The **standard deviation** is the square root of variance:

$$\sigma_A = \sqrt{\sigma_A^2}.$$

For a discrete random variable, replace the integrals by sums.

Note: When we work with data, we cannot compute these integrals directly, since they assume we know the underlying distribution. What we compute in practice, using long vectors of actual data, are **temporal averages**. If we have a property called *ergodicity*, temporal averages will converge to the statistical averages, when the number of samples goes to infinity.

2.2 Temporal mean, variance, and standard deviation

Temporal averages are what we compute in practice, e.g. in Matlab, when we have a finite vector of samples.

For a finite vector $x[1], \ldots, x[N]$, its **temporal mean**, also known as the **sample mean**, is

$$m = \bar{X} = \frac{1}{N} \sum_{n=1}^{N} x[n].$$

The **temporal variance**, also known as the **sample variance**, is:

$$v = \frac{1}{N-1} \sum_{n=1}^{N} (x[n] - \bar{X})^2$$

or:

$$v = \frac{1}{N}\sum_{n=1}^{N}(x[n] - \mu_X)^2$$

The difference between is the two expressions in the first term, known as the **normalization factor**, which is either $\frac{1}{N-1}$ or $\frac{1}{N}$. The choice depends on whether the mean μ_X is estimated from the same data or is known *a priori*:

- use $\frac{1}{N-1}$ when the mean μ_X is estimated from the same data;
- use $\frac{1}{N}$ when the mean μ_X is the true mean, and is known *a priori*.

The **temporal (or sample)standard deviation** is the square root of variance:

$$s = \sqrt{v}.$$

For an infinite-length sequence, the temporal averages are defined as a limit of averages over increasingly long intervals.

2.3 Computing averages in Matlab

For a vector, `mean()` computes the arithmetic mean. `var()` and `std()` default to normalization by $N-1$, whereas `var(x,1)` and `std(x,1)` use N:

```
x = [1, 2, 1, 5, 6, 5, 6, 6, 7, 10, 14, -3];
m = mean(x)
v = var(x)       % denominator N-1
s = std(x)       % square root of var(x)
vN = var(x, 1)   % denominator N
sN = std(x, 1)   % square root of var(x, 1)
```

For a matrix `X`, the functions `mean(X)`, `std(X)`, and `var(X)` operate **along the columns** by default, giving one result per column.

In order to perform the averages along rows, specify the dimension argument `dim=2` (see documentation). For example, for a matrix `X` of size `N`-by-`M`, the following commands compute temporal and ensemble averages:

```
% mean(X, dim)
mean(X)     % Operate on columns. Produces a 1-by-M vector of means along columns
mean(X, 2)  % Operate on rows. Produces a N-by-1 vector of means along rows
% Same for var()
% Use like this: var(X, normalization, dim)
var(X)         % Uses 1/(N-1), operates on columns
var(X, 1)      % Uses 1/N, operates on columns
var(X, 0, 2)   % Uses 1/(N-1), operates on rows
var(X, 1, 2)   % Uses 1/N, operates on rows
% Same for std()
std(X)         % Uses 1/(N-1), operates on columns
std(X, 1)      % Uses 1/N, operates on columns
std(X, 0, 2)   % Uses 1/(N-1), operates on rows
std(X, 1, 2)   % Uses 1/N, operates on rows
```

2.4 Loading variables from a file

`load()` reads variables saved in a `.mat` file:

```
D = load('data.mat');   % Store the saved variables in a structure
x = D.x;                % Access the variable named x

load('data.mat');       % Load the variables directly into the Workspace
```

With an output argument, the variables become fields of a **structure**, accessed using a dot. Without an output argument, they appear directly in the Workspace and can overwrite variables with the same names. The file must be in the current folder or specified by its path.

2.5 Summary: functions and notation you should know

Signature or notation	Example	Description
$\mathrm{E}\{A\}$	$\mu_A = \mathrm{E}\{A\}$	Statistical mean, defined by the distribution
$\mathrm{E}\{A^2\}$	$\overline{A^2} = \int_{-\infty}^{\infty} x^2 w_A(x)\,dx$	Average squared value (second moment), not μ_A^2
σ_A^2, σ_A	$\sigma_A = \sqrt{\sigma_A^2}$	Statistical variance and standard deviation

Signature or notation	Example	Description
`mean(x)`	`mean(x)`	Arithmetic mean of a vector; temporal mean if entries are time samples
`std(x,1)`, `var(x,1)`	`var(x,1)`	Observed spread normalized by N
`std(x)`, `var(x)`	`std(x)`, `var(x)`	Sample standard deviation and variance, normalized by $N-1$
`mean(X,dim)`	`mean(X,2)`	Average along the chosen matrix dimension
`std(X,w,dim)`, `var(X,w,dim)`	`std(X,0,1)`	Spread along a dimension: `w=0` uses $N-1$, `w=1` uses N
`X(:)`	`mean(X(:))`	All matrix entries as one column vector
`load(filename)`	`load('data.mat')`	Load saved variables directly into the Workspace
`D = load(filename)`	`D = load('data.mat'); x = D.x;`	Load saved variables into a structure; access fields using a dot

3 Theoretical Exercises

1. Let A be a random variable uniformly distributed on $[a, b]$, where $a < b$.

   Using the definitions of the statistical averages, compute in terms of a and b:

   a. The statistical average value $\mu_A = \mathrm{E}\{A\}$.
   b. The average squared value $\mathrm{E}\{A^2\}$.
   c. The variance $\sigma_A^2 = \mathrm{E}\{(A - \mu_A)^2\}$.

   Special case: $a = -4$, $b = 6$.

4 Matlab Exercises

Write all commands yourself in a script or live script. Label figures and answer interpretation questions in comments. Run the script from `Labs/` so that the relative dataset paths work.

1. Mean, standard deviation, and variance:

   a. Generate a vector `x` of $N = 1000$ random values from $\mathcal{N}(\mu = 3, \sigma^2 = 4)$, using `randn()` and the scaling and shifting learned in Lab 3.
   b. Compute mean, standard deviation and variance from the vector, and compare them with the theoretical mean, standard deviation, and variance. Why are they not exactly equal?
   c. Run everything again (re-generate the vector, recompute the statistics, and compare the results). Do the results change? Why?
   d. Compute the mean manually using `sum()` and `numel()`, then compare with `mean()`.
   e. Check that `std(x)^2` agrees with `var(x)` to numerical precision.
   f. Say what are the mean, standard deviation, and variance of `x+10` and `2*x`. Which quantities change when shifting? And when scaling?
   g. Verify your predictions by actually computing `x+10` and `2*x`, and then computing their mean, standard deviation, and variance.

2. Election turnout:

   Load `ElectionsData.mat`, containing data from the local elections in Iași on 27 September 2020:

```
D = load('ElectionsData.mat');
values = double(D.values);
names = D.names;
```

`double()` converts the stored integer counts to floating-point values before calculations.

Each row of `values` corresponds to one voting centre. The cell array `names` contains the corresponding names in the same order; `names{k}` gives the name of the centre in row `k`. The columns of `values` are:

Column	Meaning
1	Registered voters on permanent lists
2	Registered voters on complementary lists
3	Votes from permanent lists
4	Votes from complementary lists
5	Votes from supplementary lists
6	Votes using the mobile ballot box

For this exercise, define turnout as:

$$\text{turnout (\%)} = 100\,\frac{\text{total votes (columns 3–6)}}{\text{registered voters (columns 1–2)}}.$$

a. Compute the total registered voters and total votes for each centre. Store them in column vectors `registered` and `votes`. Hint: use `sum()` and specify the dimension along the rows.
b. Make sure that all entries of `registered` and `votes` are positive. Write the code to make all negative entries, if there are any, equal to zero.
c. Compute the turnout percentage for each centre (use `./`).
d. Identify the names of the centres with maximum and minimum turnout. Report their turnout percentages. Hint: use the second output of `max()` and `min()` to obtain the indices, then use these to access `names`.
e. Plot turnout against centre index. Plot a histogram of turnout.
f. Compute the mean, standard deviation, and variance of turnout. Express the standard deviation in percentage points.
g. Identify potential outliers in the turnout vector using the three-sigma rule: flag any value strictly outside $[m - 3s,\ m + 3s]$, where m is the mean turnout and s is its sample standard deviation (use `std()`). Use logical indexing to extract the flagged turnout percentages and the corresponding centre names. Treat these as potential outliers, not necessarily errors.
h. Compute the turnout for the entire city: total votes across all centres divided by total registered voters, multiplied by 100. Compare it with the mean from point f. Why are they different?
i. Divide the centres into two groups:
   - those with registered voters fewer than `median(registered)`,
   - those with registered voters equal or more than `median(registered)`.

Compute the mean and standard deviation in each group. Which group has greater spread?

3. ECG signals:

   Load `ECG_test.mat`, containing 80 ECG signals:

   ```
   E = load('ECG_test.mat');
   X = E.test;
   ```

   Each column of `X` contains one signal of 256 samples. Each row contains the values from all signals at the same sample index. Use sample index on the horizontal axis and amplitude in dataset units on the vertical axis.

   a. Extract the first signal, i.e. the first column of X, and store it in a variable `s`. Plot it. Compute its mean, standard deviation, and variance.
   b. Divide `s` into four consecutive blocks of 64 samples. Compute the mean and standard deviation of each block. Which block has the greatest standard deviation? Compare with the plot.
   c. Compute the temporal mean, standard deviation, and variance of every signal. Store the results in three row vectors of 80 entries. Hint: Call the functions just once, and specify which dimension to operate along.
   d. Plot the temporal means and standard deviations against signal index in separate figures. Do all signals have similar means? Similar spread?
   e. Compute the mean and standard deviation across the 80 signals at each sample index. Store the results in two column vectors of 256 entries. Hint: specify the dimension along the rows.
   f. Plot the mean signal from point e.
   g. Plot signals 1, 2, and 3 together with the mean signal (use the `plot()` function with a matrix of data, check doc). Do their peaks occur at similar sample indices?

5 Final questions

1. Randomly reorder the samples of an ECG signal. What happens to its mean and variance? Do they change or not?
2. Split one voting centre into two identical halves, dividing both its votes and registered voters equally. Can the mean centre turnout change even though nobody's vote has changed? What happens to the city turnout?
3. Independently shift each ECG signal by a different number of samples, wrapping samples around the ends. Each signal keeps its mean and variance. Does the mean signal across records stay the same?